

StegaQR: Learned Private Data Embedding in Standard-Decodable QR Codes

Anonymous submission

Abstract: Quick Response (QR) codes expose one public payload to any reader. StegaQR adds a private bitstring through learned color perturbations while preserving the public payload for a conventional reader. Its spatial bit-grid assigns each coded bit to a cell that is upsampled to the symbol resolution. Under the same training budget, this layout reaches full recovery at 100 bits while global message broadcast remains near chance. Three embedding modes provide channel isolation, joint color placement, or exact protection of QR structural modules. Across five seeds, models trained through a stochastic photometric distortion proxy achieve 99.1 to 100 percent raw full-codeword recovery under that proxy. On 11 held-out, non-differentiable operators, including true JPEG and a combined resize, blur, brightness, noise, and JPEG pipeline, repetition-coded message recovery is 100 percent in all 256 trials per operator. The complete detector, rectifier, and neural decoder pipeline also recovers both payloads from all ten phone photographs in a physical pilot. We report the capacity and imperceptibility tradeoff, an architecture ablation, and a classical baseline.

Index Terms: QR codes, information hiding, data embedding, deep learning, robust watermarking, error-correction coding, screen-shooting robustness.

I. INTRODUCTION

A QR code carries a single public payload that any camera can read. That openness drives its use in ticketing, payments, and product labeling, but it also means the code cannot carry anything that should travel with the artifact and stay out of reach of an ordinary scanner. A second, private channel that rides inside the same printed or displayed code would support uses such as tamper evidence, device-pairing secrets, and provenance tags, provided two conditions hold. A standard reader must still decode the public payload, and the private channel must survive the way QR codes are actually consumed, which is by photographing them from paper or a screen.

Two bodies of work come close, but each leaves a gap. Neural data-hiding methods learn an encoder and decoder jointly and can embed a bitstring that survives printing and re-photography [1], [2], [3], [4]. They hide data inside natural photographs, however, so the recovered medium is the photo rather than a still-scannable code. Methods that hide data specifically inside QR codes manipulate modules, error-correction redundancy, segment choices, or color layers by hand [5], [6], [7], [8], [9]. Several preserve a public payload for a standard reader, but none of the reviewed QR-specific methods trains an encoder and decoder jointly. Their evaluation also differs from the screen-to-camera pipeline considered here.

We present StegaQR, a method that bridges these two lines. A convolutional encoder takes a standard QR code and a private bitstring and outputs a stego code that differs from the original only by a learned color perturbation. A convolutional decoder recovers the private bits from the stego code, while any conventional reader recovers the public payload unchanged. The encoder and decoder are trained end to end through a differentiable layer that simulates photometric corruption, compression noise, and blur. A standard QR detector handles localization and perspective rectification before neural decoding.

The design choice that makes this work at useful payload sizes is how the private bits enter the network. Prior neural hidings broadcast the message as a constant signal over the whole image and read it back by global pooling. We instead lay the bits out spatially. Each bit occupies one cell of a grid that is upsampled to the code resolution, and the decoder pools its activation map back to the same grid to read each cell. Because a single shared convolution then learns one write-and-read operation that applies to every cell, learning a 100-bit code becomes about as hard as learning a handful of bits. Under an identical training budget, the spatial layout reaches 100 percent bit accuracy at 100 bits, while the broadcast layout stays near 52 percent, which is chance.

StegaQR exposes three switchable embedding modes that trade fault isolation, capacity, and structural guarantees. A segregated mode uses independent per-channel sub-networks, a cross-channel mode uses all three color channels jointly, and a QR-anchored hybrid masks the perturbation off the finder, timing, and format modules so the structural anatomy of the code is provably unchanged. On top of the learned channel we apply error-correction coding, which turns the decoder's high but imperfect bit accuracy into exact message recovery. The need for distortion-aware training is concrete. Machine-learning decoders for printed or displayed 2D barcodes are brittle to real photometric and environmental variation, because a captured code depends heavily on the device and lighting, and a decoder trained for one setting often fails in another [10]. Training through the distortion layer removes this brittleness, and we confirm the result on real captures rather than on simulation alone.

Because the embedding strength is a tunable knob, StegaQR spans an operating frontier. At high peak signal-to-noise ratio (PSNR) the perturbation is not visible to a human and the embedding is genuinely covert. At the lower PSNR needed to survive physical capture the perturbation appears as a faint color tint, so we describe that regime as robust data

embedding rather than as human-imperceptible steganography. We characterize both ends and state the trade-off plainly.

This paper makes the following contributions.

- A neural method that embeds an arbitrary private bitstring inside a standard-decodable QR code by learned color perturbation, distinct from hiding data in natural images and from classical QR data hiding.
- A spatial bit-grid payload layout, with an ablation showing it is the component that makes the problem learnable at 100 bits (100 percent versus 52 percent for global broadcast).
- Three switchable embedding modes, including a QR-anchored hybrid that leaves the structural modules untouched, compared across five seeds.
- A distortion-trained, error-correction-coded channel characterized along the imperceptibility, robustness, and capacity axes, with a classical baseline for reference.
- Validation on real screen-to-camera capture, where the private message is recovered from every test photograph while the public payload stays fully standard-decodable.

II. RELATED WORK

A. Neural data hiding in images

Deep data hiding trains an encoder and a decoder as a pair. Baluja hid a full color image inside a same-size cover and recovered it with a companion network [1]. HiDDeN moved to bitstring payloads and introduced the now-standard recipe of inserting a differentiable noise layer between encoder and decoder, so the system learns robustness to blur, cropping, and an approximation of JPEG [2]. StegaStamp carried this to the physical world: it embeds a hyperlink bitstring into a natural photograph that stays decodable after the photo is printed and re-captured, using simulated printing and reimaging during training and a BCH code to clean up residual bit errors [3]. RoSteALS placed the payload as a single offset in the latent space of a pretrained autoencoder [4], and RivaGAN added attention and adversarial critics for video [11]. Two properties separate all of these from our setting. The cover is a natural image, so the artifact that must stay useful is a photograph rather than a scannable code, and the payload is injected by broadcasting the message across the whole image and reading it back with global pooling, with no spatial correspondence between a bit and a location.

B. Data hiding inside QR codes

A separate line keeps the QR code itself as the carrier. Lin and Chen embed a private payload by exploiting QR redundancy while an ordinary scanner returns only the cover content [5]. Koptyra and Ogiela choose nonoptimal data segments to encode a secret without consuming the QR error-correction budget; the resulting symbols remain valid under several readers and printed tests [6]. The two-level QR code replaces black modules with textured patterns recovered by correlation and Reed-Solomon decoding [7]. HiQ superposes monochrome codes in color and uses a trained classifier to separate them, so the composite is not a standard monochrome

QR [8]. Halftone QR binds the center of each submodule to the decoded value while using the remaining area for a visible image [9]. These methods establish that a QR symbol can carry private information, but their embedding rules are designed manually. StegaQR instead learns the visual perturbation and its decoder jointly against a degraded image channel.

C. Robustness to the physical channel

Robustness to printing and re-photography, and to the related screen-shooting channel in which a displayed image is captured by a camera, has become its own topic. Fang et al. characterized the screen-shooting channel through lens deformation, light-source deformation, and moire, and embedded the watermark redundantly at small scale to survive it [12]. PIMoG showed that a noise layer modeling only the dominant distortions, namely perspective, illumination, and moire, is enough to train a deep watermarking network end to end and reach high screen-shooting accuracy [13]. We adopt the same principle of a differentiable distortion layer during training and apply it to a QR cover.

D. Positioning

StegaQR sits at an intersection that none of these occupy. It is a learned encoder and decoder, like the neural image hid-ers, but its cover is a QR code that remains standard-decodable, like the classical QR methods. It lays the payload out spatially rather than broadcasting it, exposes switchable embedding modes including one that provably preserves the structural modules, and is validated on real screen-to-camera capture. The closest neural neighbor is StegaStamp, which is physical but uses natural-image covers. The closest QR neighbors are the classical redundancy, segmentation, and textured module schemes, which preserve a public payload without learning the embedding channel. Table I summarizes these contrasts.

III. METHOD

A. Overview

StegaQR turns a public payload and a private bitstring into a single stego QR image (Fig. 1). The public payload is rendered as a standard QR code that serves as the cover. The private bitstring is protected by an error-correcting code, arranged on a spatial grid, and fused with the cover by a convolutional encoder that outputs a bounded color perturbation. At extraction, a standard reader recovers the public payload from the stego image directly, while a convolutional decoder recovers the private bits. The encoder and decoder are trained jointly with a differentiable distortion layer placed between them.

B. Cover and payload representation

Cover. The public payload is encoded as a version-4 QR code with error-correction level M under ISO/IEC 18004 [14], which gives a 33×33 module grid. Each module is rendered as a square of m pixels, so the cover is an RGB image of side $33m$; we use $m = 4$. Dark modules are black and light modules are white.

TABLE I
COMPARISON WITH REPRESENTATIVE NEURAL HIDING AND QR-SPECIFIC METHODS. PUBLIC QR INDICATES THAT A CONVENTIONAL MONOCHROME READER STILL RETURNS THE PUBLIC PAYLOAD.

Method	Cover	Learned	Payload	Imperceptibility	Robustness validated	Public QR
HiDDeN [2]	natural image	yes	52 b (0.203 bpp)	visually indist.	simulated	n/a
StegaStamp [3]	natural photo	yes	56 b (post-BCH)	near-invisible	print + photo	n/a
Lin/Chen [5]	QR	no	cover-dependent	module changes	digital	yes
2LQR [7]	QR	no	up to 20,000 b	visible (textured)	print-scan	yes
Segmentation [6]	QR	no	segment choices	valid QR	digital + print	yes
HiQ [8]	color QR	ML dec.	up to 8,900 B	overt color	mobile capture	no
StegaQR (ours)	QR	yes	100 b raw, 33 to 56 b net	18 to 55 dB	proxy + held-out + camera	yes

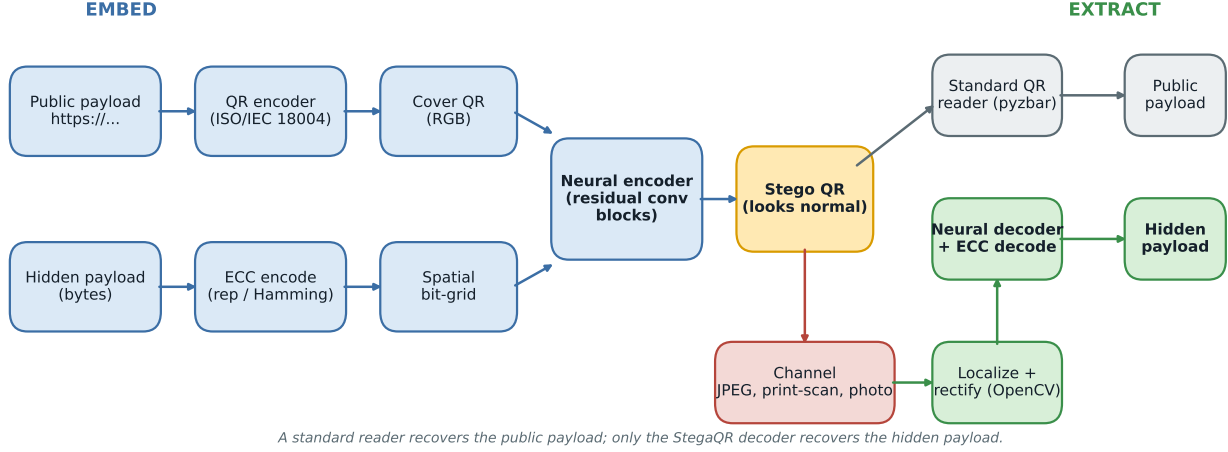


Fig. 1. StegaQR pipeline. The public payload is rendered as a standard QR cover; the private bitstring is error-correcting coded, laid out on a spatial bit-grid, and fused with the cover by a convolutional encoder that outputs a bounded color perturbation. A standard reader recovers the public payload from the stego code, while the trained decoder recovers the private bits. Encoder and decoder are trained jointly through a differentiable distortion layer.

Payload. The private message is first expanded by an error-correcting code (Section III-G) into L coded bits. We place these bits on a square grid. Let $g = \lceil \sqrt{L} \rceil$. The bits fill a $g \times g$ grid, one bit per cell, and the grid is upsampled by nearest-neighbor interpolation to the cover resolution to form a single-channel bit map $B \in \{0, 1\}^{H \times W}$. Each bit thus occupies a contiguous block of pixels at a fixed location.

C. Spatial bit-grid

The bit map is the reason the task becomes learnable. A convolution is translation equivariant with shared weights, so a network that learns to write one cell has learned to write every cell, and likewise for reading. Embedding L bits is then about as hard as embedding a few. This is the point of departure from the global broadcast used by prior neural hiders [2], [3], [4], which replicate the message uniformly over the image and read it back by global pooling; there the network must learn L distinct global patterns, which does not scale to the payload sizes we need. Section V-B quantifies the gap. Figure 2 shows the write and read paths.

D. Encoder and decoder

Encoder. The encoder receives the concatenation of the cover, the bit map, and, in the hybrid mode, a structural mask

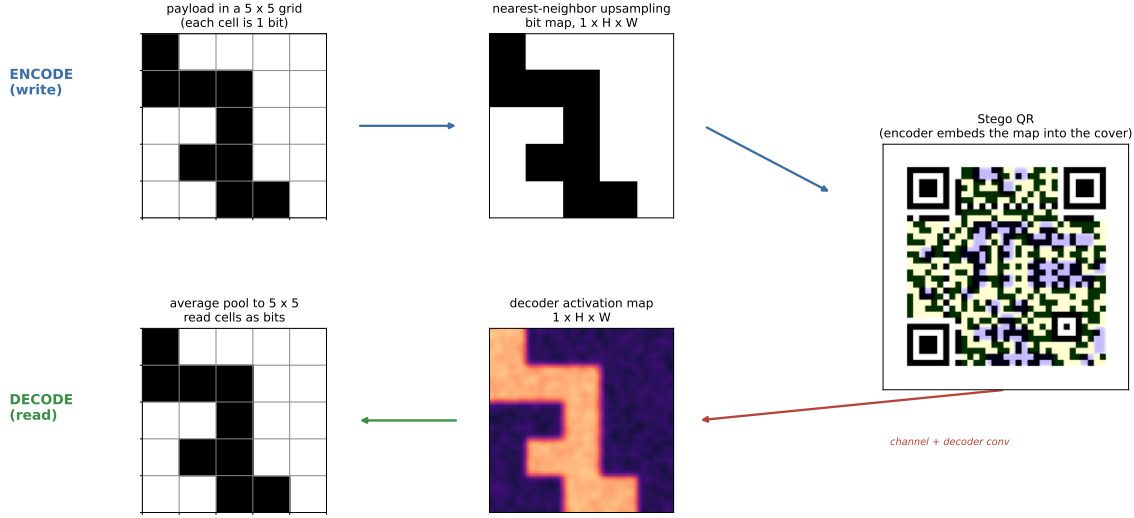
(Section III-E). It applies five 3×3 convolutional blocks with 64 channels, each block being convolution, group normalization, and ReLU with a residual connection, followed by a 1×1 convolution to three channels and a tanh nonlinearity. The output is scaled by a perturbation bound ε , added to the cover, and clamped:

$$\hat{x} = \text{clip}(x + \varepsilon \tanh(f_\theta(x, B)), 0, 1). \quad (1)$$

The bound ε is the single knob that trades imperceptibility against robustness.

Decoder. The decoder applies six convolutional blocks to the stego image, a 1×1 convolution to a single-channel activation map, an adaptive average pool to the $g \times g$ grid, and a per-cell readout that yields L logits. Thresholding gives the recovered coded bits, which the error-correcting code maps back to the message. The architecture is shown in Fig. 3.

Normalization. We use group normalization rather than batch normalization. In the robust regime the perturbation is small relative to the cover, and the running statistics that batch normalization accumulates during training do not match the per-batch statistics seen at inference, which collapses decoding at test time. Group normalization removes this train and test mismatch.



A shared convolution learns the same write and read operation at every cell. At 100 bits the spatial layout reached full recovery, while global broadcast remained near chance.

Fig. 2. Spatial bit-grid. On embedding (top) the payload fills a $g \times g$ grid, one bit per cell, upsampled to the code resolution and written into the cover by the encoder. On extraction (bottom) the decoder produces an activation map, average-pools it back to the grid, and reads each cell. A single shared convolution learns one write-and-read operation that applies to all cells.

E. Channel modes

The three modes share the template above but differ in how color is used and what is protected.

Segregated. Three independent sub-encoders and sub-decoders, one per color channel, each embedding $L/3$ bits on its own grid. A failure in one channel cannot corrupt the others, which gives fault isolation at the cost of treating the channels separately.

Cross-channel. A single encoder and decoder use all three channels jointly, which gives the most freedom in placing the perturbation.

Hybrid (QR-anchored). The cross-channel design with two QR-aware constraints. A binary structural mask marks the finder, timing, format, and alignment modules, and the encoder multiplies its perturbation by this mask so those modules are left exactly as in the cover, which guarantees the structural anatomy of the code is unchanged. Because masked cells cannot carry payload, the hybrid layout places bits only on the data-rich cells of a finer grid, selected once from the mask. We found that an explicit color-calibration branch in the decoder destabilized training without improving accuracy, so the hybrid decoder omits it; the convolutional decoder already absorbs photometric variation.

F. Distortion layer and training

Distortion layer. A differentiable layer sits between the encoder and decoder during training. It applies additive Gaussian noise, a quality-conditioned additive proxy for compression error, brightness scaling, per-channel color shift, and Gaussian blur, each with a fixed probability. The proxy does not

perform a discrete cosine transform. True JPEG and other non-differentiable operators are reserved for held-out evaluation.

Objective. The loss combines a decoding term, an imperceptibility term, and a structural term:

$$\mathcal{L} = \lambda_d \mathcal{L}_{\text{dec}} + \lambda_p \mathcal{L}_{\text{perc}} + \lambda_s \mathcal{L}_{\text{struct}}. \quad (2)$$

$\mathcal{L}_{\text{dec}}$ is the binary cross-entropy between recovered and true coded bits. $\mathcal{L}_{\text{perc}}$ combines mean squared error with a penalty on the largest per-pixel deviation, keeping the perturbation small. $\mathcal{L}_{\text{struct}}$ is a hinge that keeps structural modules close to their cover values. The hybrid decoder also uses an auxiliary confidence loss with weight $\lambda_c = 0.1$; it does not affect the payload logits used for the reported recovery metrics.

Curriculum. Optimizing the decoding and imperceptibility terms together from scratch is unstable: the imperceptibility term can drive the perturbation to zero before the decoder has learned to read it. We therefore train decoding only for a short warmup, then ramp the imperceptibility and structural weights linearly to their targets. The networks are optimized with Adam at a base rate of 10^{-3} under cosine annealing with gradient clipping. Checkpoint selection maximizes the sum of raw full-codeword recovery and bit accuracy under stochastic proxy distortion, with a small PSNR tie-break. This avoids both a high-PSNR solution that loses bits and an unnecessarily strong perturbation.

G. Error-correction coding

The decoder recovers individual bits with high but imperfect accuracy, and one wrong bit ruins a message. We therefore protect the message with an error-correcting code before

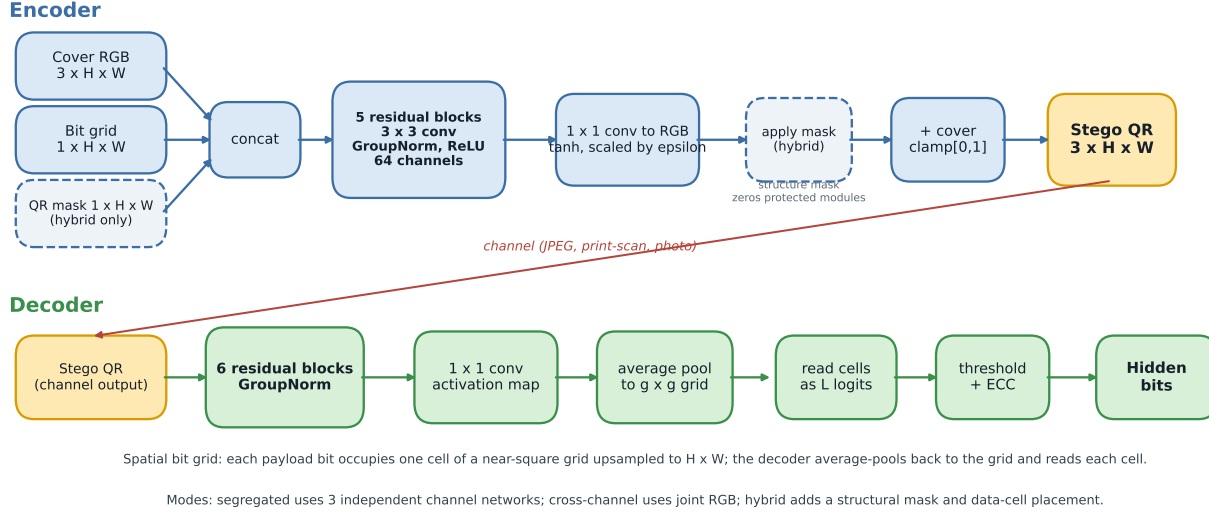


Fig. 3. Encoder and decoder. The encoder fuses the cover, the bit-grid map, and (hybrid only) the structural mask through convolutions with group normalization and emits a bounded, optionally masked perturbation added to the cover. The decoder reads the payload by pooling its activation map back to the bit-grid. Group normalization is used throughout.

embedding. We use a repetition code with soft-decision combining, in which the decoder sums the logits of the repeated copies, and a Hamming(7,4) code as a higher-rate alternative. With k message bits expanded to L coded bits, the net capacity is k and the rate is k/L . Coding converts the decoder’s per-bit accuracy into exact message recovery at the cost of capacity.

IV. EXPERIMENTAL SETUP

Data. Training data is generated on the fly. Each sample is a version-4, level-M QR code rendered from a random alphanumeric public payload, paired with a random private message. No external dataset is used. Unless stated otherwise the payload is $L = 100$ coded bits and the module size is $m = 4$, giving a 132×132 cover. The segregated mode uses 99 bits so its payload divides evenly across three channels.

Training. Each configuration is trained for five seeds. We train for 24 epochs with batch size 16, 1500 fresh samples per epoch, Adam at 10^{-3} with cosine annealing and gradient clipping, a three-epoch decode-only warmup, and an eight-epoch ramp of the imperceptibility and structural weights. We compare two regimes: a clean regime with the distortion layer disabled, and a distortion-trained regime with it enabled. Unless noted, the perturbation bound is $\varepsilon = 0.3$ and the encoder adapts the actual perturbation downward through the imperceptibility loss.

Metrics. We report bit accuracy (fraction of correct coded bits), full-decode rate (fraction of samples with every coded bit correct), message-decode rate (fraction of messages recovered exactly after error correction), PSNR and SSIM of the stego image against the cover, and public decode rate, the fraction of stego images whose public payload a standard reader (ZBar) decodes correctly. Rates are reported as mean and a 95 percent confidence interval over the five seeds.

Held-out operators. Proxy-distorted evaluation uses the stochastic layer used during training. We separately test true JPEG, PIL Gaussian blur, resize round trips, brightness changes, sensor noise, and a combined pipeline. These operators do not participate in gradient training.

Hardware. All experiments run on a single 16 GB GPU.

V. RESULTS

A. Channel modes and the effect of distortion training

Table II reports the three modes under both training regimes. Two patterns are consistent across seeds. First, the public payload is decoded by a standard reader in every condition, so the embedding never breaks the cover. Second, distortion training produces the robustness. Clean-trained models are near perfect on clean stego images but recover the full codeword from only about 10 to 14 percent of proxy-distorted images, whereas distortion-trained models recover it from 99 to 100 percent. Cross-channel and segregated behave almost identically; the hybrid mode matches them at a slightly higher PSNR, because restricting the perturbation to data modules concentrates it where it is useful.

B. The spatial bit-grid is the enabling component

To isolate the bit-grid, we replace it with the global broadcast used by prior neural hidere and train both under an identical budget. At 100 bits the spatial grid reaches 100 percent bit accuracy and 100 percent full-decode, clean and distorted, while the broadcast layout stays at 51 to 53 percent bit accuracy, at chance level, and never achieves a single full decode. At this payload size the grid is therefore a prerequisite for learning rather than a refinement of it.

TABLE II
CHANNEL MODES WITH 99 BITS FOR SEGREGATED AND 100 BITS OTHERWISE, FIVE SEEDS (MEAN $\pm$ 95% CI). FDR REQUIRES EVERY CODED BIT TO BE CORRECT.

Mode	Training	Clean bit-acc	Clean FDR	Proxy bit-acc	Proxy FDR	PSNR (dB)	Public decode
Segregated	clean	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	65.5 $\pm$ 5.7	13.9 $\pm$ 11.3	63.5	100.0
	distortion	100.0 $\pm$ 0.0	99.8 $\pm$ 0.3	100.0 $\pm$ 0.0	99.1 $\pm$ 0.8	17.6	100.0
Cross-channel	clean	100.0 $\pm$ 0.0	99.5 $\pm$ 0.9	63.8 $\pm$ 10.6	10.0 $\pm$ 19.6	61.7	100.0
	distortion	100.0 $\pm$ 0.0	99.8 $\pm$ 0.3	100.0 $\pm$ 0.0	99.2 $\pm$ 1.5	19.0	100.0
Hybrid	clean	100.0 $\pm$ 0.0	99.8 $\pm$ 0.3	66.1 $\pm$ 7.1	13.9 $\pm$ 11.3	58.8	100.0
	distortion	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	22.3	100.0

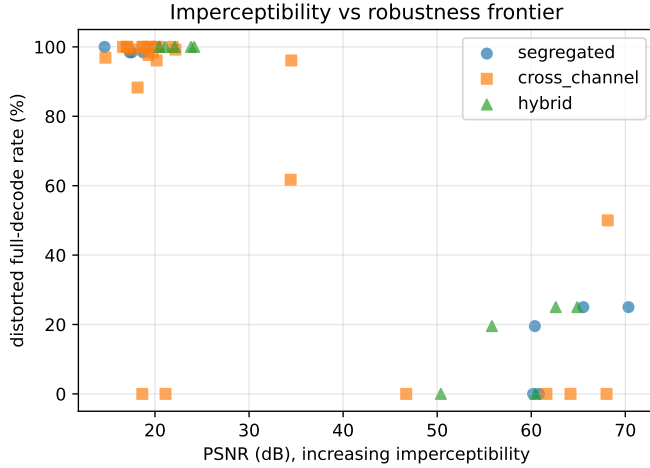


Fig. 4. Imperceptibility against robustness across trained models. The covert regime (high PSNR) is near invisible; the robust regime (lower PSNR) survives capture.

C. Imperceptibility and robustness form a tunable frontier

The perturbation bound ε moves the system along a frontier (Fig. 4). At small ε the stego code is visually indistinguishable from the cover. A clean-trained cross-channel model reaches 54.9 dB PSNR with SSIM 1.000 and full public decode, which is the covert regime (Fig. 5, second column). Surviving physical capture requires a larger perturbation, which appears as a faint color tint at PSNR near 20 dB; this is the robust regime that we use for the capture study. Figure 5 also shows that the hybrid residual is exactly zero on the finder patterns, confirming that the structural modules are untouched.

D. Capacity scaling

Table III sweeps the embedded capacity of the cross-channel model under distortion. Distorted bit accuracy stays at or above 99 percent from 25 to 200 bits, and the full-decode rate stays high except at 150 bits, where one seed was an outlier and widened the interval. Higher capacity is therefore available, with the usual caveat that exact full-message decode at large payloads benefits from error correction.

E. Error-correction coding gives exact messages

A single wrong bit ruins a message, so we measure message-decode rate after coding (Table IV). On the

TABLE III
CAPACITY SCALING, CROSS-CHANNEL, TRAINED AND EVALUATED WITH PROXY DISTORTION.

Capacity (bits)	Dist. bit-acc	Dist. FDR	PSNR (dB)
25	99.9 $\pm$ 0.1	97.7 $\pm$ 2.3	23.7
50	100.0 $\pm$ 0.0	99.7 $\pm$ 0.5	20.3
100	100.0 $\pm$ 0.0	99.2 $\pm$ 1.5	19.0
150	99.3 $\pm$ 1.4	86.5 $\pm$ 24.2	23.9
200	100.0 $\pm$ 0.0	95.3 $\pm$ 7.0	19.0

TABLE IV
MESSAGE-DECODE RATE UNDER PROXY DISTORTION (CROSS-CHANNEL, 100 EMBEDDED BITS).

Code	Net bits	Rate	Message decode
None (raw)	100	1.00	99.2 $\pm$ 1.5
Repetition-3	33	0.33	99.5 $\pm$ 0.6
Repetition-5	20	0.20	98.1 $\pm$ 1.1
Hamming(7,4)	56	0.57	100.0 $\pm$ 0.0

distortion-trained cross-channel model, a Hamming(7,4) code recovers the message exactly in every case at 56 net bits, and a rate-1/3 repetition code reaches 99.5 percent at 33 net bits. Coding cleanly converts the decoder's high bit accuracy into exact recovery.

F. Classical baseline

A classical per-channel least-significant-bit scheme embeds the same payload with no training. It is perfect on clean images, at 100 percent full-decode and 78 dB PSNR, but collapses under proxy distortion to 53.8 percent bit accuracy and 3.9 percent full-decode. The learned, distortion-trained channel closes this gap.

G. Held-out non-differentiable operators

We evaluate the released hybrid checkpoint on 256 fresh messages under each of 11 operators that are not used for gradient training. Repetition-coded message recovery is 100 percent for every operator. Raw full-codeword recovery is 100 percent for JPEG quality 70, blur radius 1, and a 50 percent resize round trip. It remains 95.3 percent for JPEG quality 30 and blur radius 2, and 94.9 percent for the combined resize, blur, brightness, noise, and JPEG pipeline. Error correction removes every residual message error in this evaluation.

Cover and stego images across the imperceptibility and robustness frontier

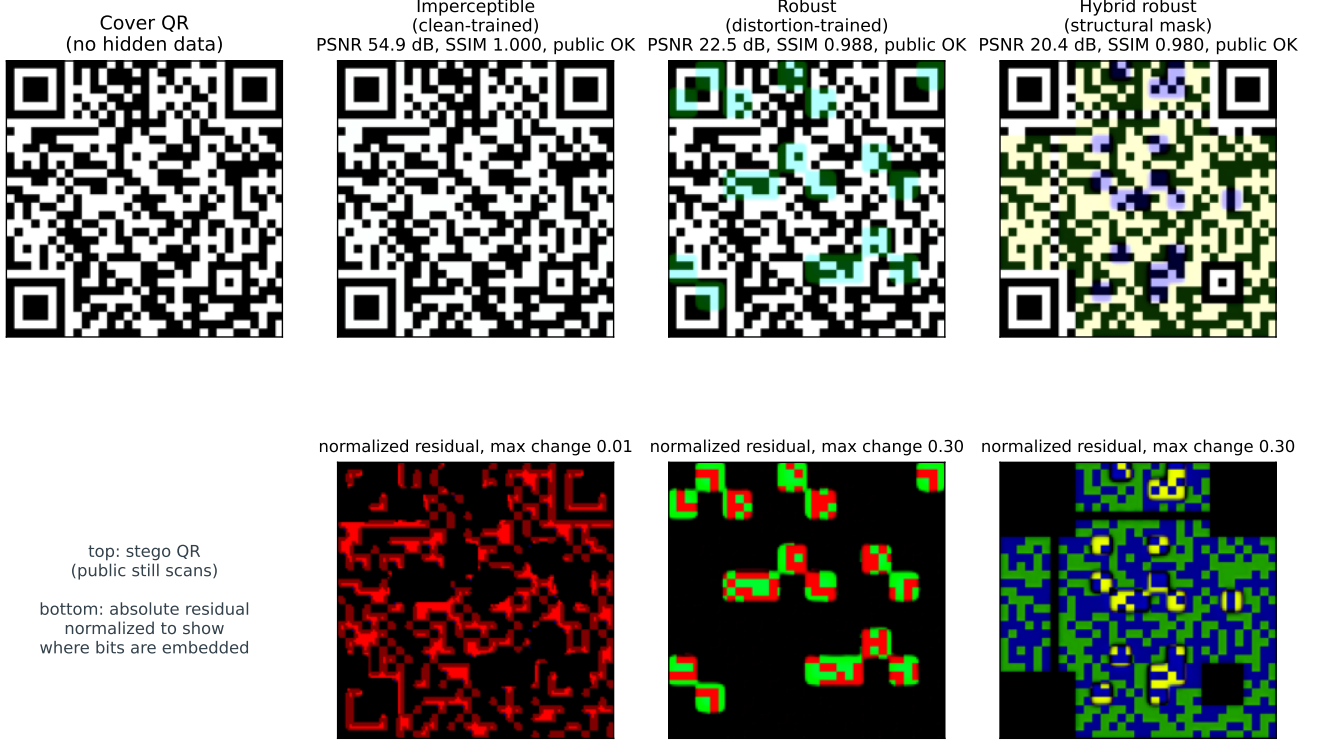


Fig. 5. Cover versus stego at three operating points with normalized residuals. The covert model is visually identical to the cover at 54.9 dB; the robust and hybrid models carry a faint tint. The hybrid residual is zero on the finder patterns (bottom right), showing the structural mask leaves them unchanged. All public payloads decode.

H. Real screen-to-camera capture

The image-operator tests do not reproduce a complete camera channel, so we also test real captures. We displayed ten distinct stego codes from the distortion-trained hybrid model on a monitor and photographed them with a smartphone, free-hand, at varied angle and distance. A standard OpenCV detector localized and rectified each photograph before the neural decoder received it. The QR was located in all ten, the public payload decoded in all ten, and the private message, protected by a rate-1/3 code, was recovered in all ten (Fig. 6). This result validates the complete detection and decoding pipeline; it does not claim that the neural decoder itself learns perspective invariance.

VI. DISCUSSION AND LIMITATIONS

The frontier makes the main limitation explicit: the covert regime, where the perturbation is invisible, does not survive physical capture, and the regime that survives capture carries a faint but visible color tint. We therefore present the visible robust regime as robust data embedding rather than as human-imperceptible steganography, and reserve the covert label for the high-PSNR end. A reader who looks closely can tell a robust StegaQR code from a plain one; what they cannot do is read the private payload.

Net capacity is modest. At 100 embedded bits a rate-1/3 code leaves 33 message bits, and a Hamming code leaves 56, which is enough for an identifier, a short token, or a key but not for a long message. The capacity sweep shows that the embedded budget extends to at least 200 bits at high bit accuracy, so larger payloads are reachable, though exact recovery at the top of that range leans more on the error-correcting code.

The evaluation fixes the QR version, error-correction level, and resolution, and the physical study is a single session on one display and one camera with ten codes. These choices are enough to establish that distortion training transfers to real capture, but a broader sweep over devices, lighting, angle, and printed media, and over QR versions, would strengthen the deployment case. We also do not study steganalysis: we do not measure how hard it is for an adversary who knows the scheme to detect or remove the payload. In the covert regime detectability is the natural next question, and in the robust regime the relevant question is removal under adversarial post-processing.

Finally, the hybrid mode needed care to train. The structural mask removes payload capacity from the protected cells, and an explicit calibration branch we initially included made the cold start unstable. Placing bits only on data-rich cells and

Physical pilot: monitor display photographed with a phone

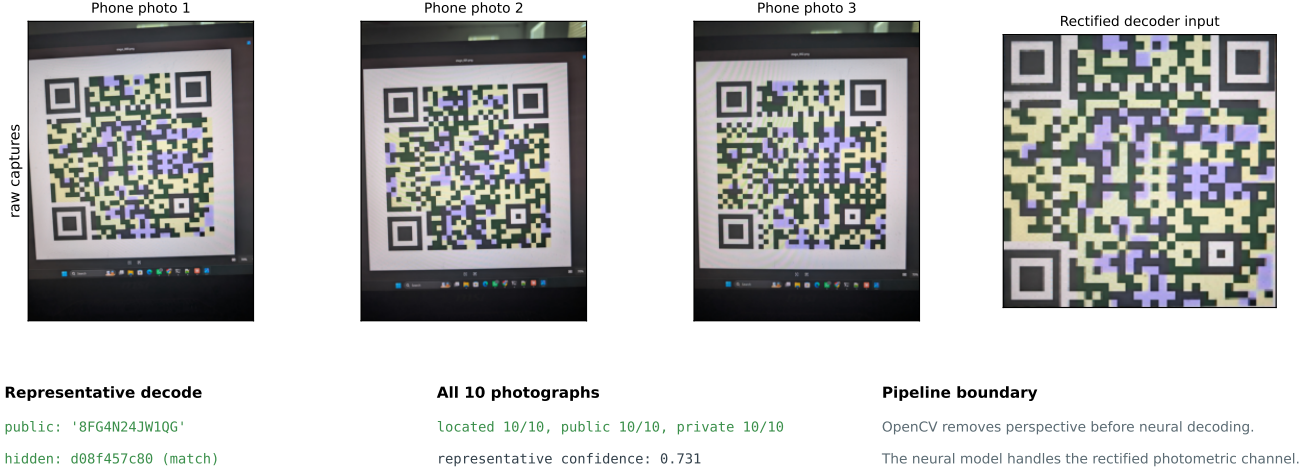


Fig. 6. Real screen-to-camera capture. Stego codes displayed on a monitor and photographed with a smartphone are localized, rectified, and decoded. Across ten photos the public payload and the private message are recovered in every case.

removing that branch resolved both, but it shows that QR-aware constraints interact with optimization in ways that a plain image-hiding setup does not face.

VII. CONCLUSION

We presented StegaQR, a learned method that embeds a private bitstring inside a standard-decodable QR code through a bounded color perturbation. The design rests on a spatial bit-grid that gives each bit a location, which lets a single shared convolution learn the embedding at useful payload sizes where the global broadcast of prior neural hidings stalls at chance. Three switchable modes offer different balances of fault isolation, capacity, and structural preservation, and a QR-anchored hybrid leaves the structural modules provably untouched. Training end to end through a differentiable distortion proxy, and coupling the channel with error correction, yields 100 percent message recovery across 11 held-out image operators and every one of ten photographs in the physical pilot, while a standard reader recovers the public payload in every case. The gains over a clean-trained model and a classical baseline are large, and the imperceptibility and robustness ends of the system are characterized as a single tunable frontier. Code and a tested package are prepared for release. Future work includes larger payloads, a broader physical and cross-device study, coverage of more QR versions, and a steganalysis of both regimes.

REFERENCES

- [1] S. Baluja, “Hiding images in plain sight: Deep steganography,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [2] J. Zhu, R. Kaplan, J. Johnson, and L. Fei-Fei, “HiDDeN: Hiding data with deep networks,” in *European Conference on Computer Vision (ECCV)*, 2018, p. 657 to 672.
- [3] M. Tancik, B. Mildenhall, and R. Ng, “StegaStamp: Invisible hyperlinks in physical photographs,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, p. 2117 to 2126.
- [4] T. Bui, S. Agarwal, N. Yu, and J. Collomosse, “RoSteALS: Robust steganography using autoencoder latent space,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2023, p. 933 to 942.
- [5] P.-Y. Lin and Y.-H. Chen, “QR code steganography with secret payload enhancement,” in *IEEE International Conference on Multimedia and Expo Workshops*, 2016, p. 1 to 5.
- [6] K. Koptyra and M. R. Ogiela, “Steganography in QR codes: Information hiding with suboptimal segmentation,” *Electronics*, vol. 13, no. 13, p. 2658, 2024.
- [7] I. Tkachenko, W. Puech, C. Destruel, O. Strauss, J.-M. Gaudin *et al.*, “Two-level QR code for private message sharing and document authentication,” *IEEE Transactions on Information Forensics and Security*, vol. 11, no. 3, p. 571 to 583, 2016.
- [8] Z. Yang, H. Xu, J. Deng, C. C. Loy, and W. C. Lau, “Robust and fast decoding of high-capacity color QR codes for mobile applications,” *IEEE Transactions on Image Processing*, vol. 27, no. 12, p. 6093 to 6108, 2018.
- [9] H.-K. Chu, C.-S. Chang, R.-R. Lee, and N. J. Mitra, “Halftone QR codes,” *ACM Transactions on Graphics (Proc. SIGGRAPH Asia)*, vol. 32, no. 6, p. 217:1 to 217:8, 2013.
- [10] N. Xie, Q. Zhang, J. Hu, G. Luo, and C. Chen, “Low-cost anti-copying 2D barcode by exploiting channel noise characteristics,” *IEEE Transactions on Multimedia*, 2021, arXiv:2001.06203.
- [11] K. A. Zhang, L. Xu, A. Cuesta-Infante, and K. Veeramachaneni, “Robust invisible video watermarking with attention,” *arXiv preprint arXiv:1909.01285*, 2019.
- [12] H. Fang, W. Zhang, H. Zhou, H. Cui, and N. Yu, “Screen-shooting resilient watermarking,” *IEEE Transactions on Information Forensics and Security*, vol. 14, no. 6, p. 1403 to 1418, 2019.
- [13] H. Fang, Z. Jia, Z. Ma, E.-C. Chang, and W. Zhang, “PIMoG: An effective screen-shooting noise-layer simulation for deep-learning-based watermarking network,” in *Proceedings of the 30th ACM International Conference on Multimedia (ACM MM)*, 2022, p. 2267 to 2275.
- [14] ISO/IEC, “ISO/IEC 18004:2015: Information technology: Automatic identification and data capture techniques: QR code bar code symbology specification,” International Standard, 2015.